

Od tej lekcji: Projektowanie baz danych. Etapy projektowania. Tworzenie modelu konceptualnego

Lekcja

Temat: Projektowanie baz danych. Etapy projektowania. Tworzenie modelu konceptualnego



Projektowanie baz danych to proces tworzenia logicznej i fizycznej struktury bazy danych, która efektywnie przechowuje, organizuje i zarządza danymi, spełniając wymagania konkretnej aplikacji lub systemu.

Kluczowym aspektem tego procesu jest świadomość, że **w bazie danych przechowujemy tylko niektóre informacje o świecie rzeczywistym**. Wybór właściwych wycinków rzeczywistości i dotyczących ich danych jest bardzo istotny — od niego zależy prawidłowe działanie bazy. Aby ten wybór był właściwy, należy wskazać informacje, które powinny być przechowywane w bazie danych, oraz określić ich strukturę.



Główne etapy projektowania

Cały proces projektowania bazy danych możemy podzielić na kilka etapów:



1. Planowanie bazy danych

- Analiza wymagań systemu i użytkowników
- Określenie, które **fragmenty rzeczywistości** mają być modelowane
- Identyfikacja **konkretnych danych** potrzebnych systemowi
- Definiowanie celów, zakresu i ograniczeń projektu
- Określenie przyszłych potrzeb rozszerzania bazy



2. Tworzenie modelu konceptualnego (diagramy ERD)

- Identyfikacja **encji** (tabel) reprezentujących wybrane byty rzeczywiste
- Określenie **atrybutów** (kolumn) opisujących te byty
- Definiowanie **relacji** między encjami

- Tworzenie diagramów ERD (Entity-Relationship Diagrams)
- **Selekcja** - co włączyć, co pominąć w modelu danych



3. Transformacja modelu konceptualnego na model relacyjny

- Przekształcenie encji na tabele
- Konwersja atrybutów na kolumny
- Zamiana relacji na klucze obce i tabele łączące
- Definiowanie typów danych dla kolumn
- Określenie ograniczeń (constraints)



4. Proces normalizacji bazy danych

- **1NF** (Pierwsza postać normalna): eliminacja powtarzających się grup
- **2NF**: eliminacja zależności częściowych od klucza
- **3NF**: eliminacja przechodnich zależności
- Ocena konieczności dalszych postaci normalnych (BCNF, 4NF, 5NF)
- Rozważenie celowej denormalizacji dla poprawy wydajności



5. Wybór struktur i określenie zasad dostępu do bazy danych

- Wybór silnika bazy danych w MySQL (InnoDB, MyISAM)
- Definiowanie indeksów dla optymalizacji zapytań
- Określenie zasad bezpieczeństwa i uprawnień
- Planowanie kopii zapasowych i recovery
- Optymalizacja struktur pod kątem wydajności



Podstawowe pojęcia w projektowaniu baz danych



Encja (Entity)

Encją jest każdy przedmiot, zjawisko, stan lub pojęcie, czyli każdy obiekt, który potrafimy odróżnić od innych obiektów (na przykład: osoba, samochód, książka, stan pogody).



Zbiór encji

Encje podobne do siebie (opisywane za pomocą podobnych parametrów) grupujemy w **zbiory encji**. W praktyce relacyjnej każdy zbiór encji staje się tabelą w bazie danych.

Przykład:

- Zbiór encji "Studenci" - zawiera poszczególne encje: student Jan Kowalski, student Anna Nowak, itd.
- Zbiór encji "Książki" - zawiera: "Pan Tadeusz", "Kamienie na szaniec", "Dzieci z Bullerbyn "

Wskazówka: Projektując bazę danych, należy precyzyjnie zdefiniować encje i określić parametry, przy użyciu których będą opisywane.



Atrybut (Attribute)

Atrybut to cecha opisująca encję. Encje mają określone cechy wynikające z ich natury. Cechy te nazywamy atrybutami.

Atrybuty encji stają się kolumnami w tabeli. Każdy atrybut reprezentuje jedną kolumnę w tabeli, która przechowuje wartości tego atrybutu dla poszczególnych encji.

Zestaw atrybutów, które określamy dla encji, zależy od potrzeb bazy danych. Nie wszystkie cechy encji muszą być przechowywane - tylko te istotne z punktu widzenia systemu.



Dziedzina (Domena)

Dziedzina to zbiór dopuszczalnych wartości atrybutu. Atrybuty encji mogą przyjmować różne wartości. Projektując bazę danych, możemy określić, jakie wartości może przyjmować dany atrybut. Zbiór wartości atrybutu nazywamy dziedziną (domeną).



Problem z projektowaniem bazy danych: Relacje wiele-do-wielu



Przykład problemu: System rezerwacji hotelu

Błędne podejście:

```
CREATE TABLE reservations (  
    reservation_id INT PRIMARY KEY,  
    guest_name VARCHAR(100),  
    room_number VARCHAR(10),  
    check_in DATE,  
    check_out DATE  
);
```

Problem: Jeden gość może mieć wiele rezerwacji, a w jednej rezerwacji może być wielu gości (rodzina, grupa). Jeśli wpisujemy wszystkich gości w jednym polu guest_name jako "Jan Kowalski, Anna Kowalska, dziecko Kowalskie", pojawiają się problemy:

1. **Redundancja danych** - ten sam gość powtarza się w wielu rezerwacjach
2. **Trudność wyszukiwania** - jak znaleźć wszystkie rezerwacje konkretnego gościa?
3. **Brak integralności** - nie ma kontroli nad poprawnością danych gości
4. **Problemy z modyfikacją** - zmiana danych gościa wymaga aktualizacji wielu rekordów



Rozwiązanie: Wydzielenie encji Gości i tabeli łączącej

Właściwe podejście:

```
-- Encja: Goście
CREATE TABLE guests (
  guest_id INT PRIMARY KEY AUTO_INCREMENT,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE,
  phone VARCHAR(20)
);

-- Encja: Rezerwacje
CREATE TABLE reservations (
  reservation_id INT PRIMARY KEY AUTO_INCREMENT,
  check_in DATE NOT NULL,
  check_out DATE NOT NULL,
  status ENUM('confirmed', 'pending', 'cancelled')
);

-- Tabela łącząca dla relacji wiele-do-wielu
CREATE TABLE reservation_guests (
  reservation_id INT,
  guest_id INT,
  is_main_guest BOOLEAN DEFAULT FALSE, -- dodatkowy atrybut relacji
  PRIMARY KEY (reservation_id, guest_id),
  FOREIGN KEY (reservation_id) REFERENCES reservations(reservation_id),
  FOREIGN KEY (guest_id) REFERENCES guests(guest_id)
);
```



Zalety tego rozwiązania:

1. **Eliminacja redundancji** - dane każdego gościa przechowywane raz
2. **Łatwe wyszukiwanie** - proste zapytania o rezerwacje danego gościa
3. **Integralność danych** - kontrola przez klucze obce
4. **Elastyczność** - możliwość dodawania dowolnej liczby gości do rezerwacji
5. **Łatwość modyfikacji** - zmiana danych gościa w jednym miejscu



Inne częste problemy i ich rozwiązania:

Problem 1: Mieszanie jednostek miary w jednej kolumnie

Błąd: price DECIMAL(10,2) bez określenia waluty

Rozwiązanie: Dodaj kolumnę currency VARCHAR(3) lub normalizuj do osobnej tabeli walut

Problem 2: Przechowywanie historii zmian w głównej tabeli

Błąd: Aktualne i historyczne dane mieszane w jednej tabeli

Rozwiązanie: Wydziel tabelę historii z timestampami i flagą `is_current`

Problem 3: Nieatomowe atrybuty (adres w jednym polu)

Błąd: `address VARCHAR(200)` zawierający ulicę, miasto, kod

Rozwiązanie: Podziel na osobne kolumny: `street`, `city`, `postal_code`, `country`

Zasada: "Jedna encja = jedna odpowiedzialność"

Najczęstszy błąd w projektowaniu to próba upchnięcia zbyt wielu konceptów w jednej encji/tabeli. Każda tabela powinna reprezentować **jeden jasno zdefiniowany byt** z rzeczywistości. Jeśli zauważysz, że:

- dane się powtarzają
- masz puste pola dla niektórych rekordów
- trudno jest modyfikować dane
- zapytania stają się skomplikowane



Tworzenie modelu konceptualnego (diagramy ERD)



Definicja modelu konceptualnego

Konceptualne projektowanie bazy danych to konstruowanie schematu danych niezależnego od wybranego modelu danych, docelowego systemu zarządzania bazą danych, programów użytkowych czy języka programowania. Jest to abstrakcyjna reprezentacja struktury danych skupiona na ich znaczeniu i relacjach, a nie na implementacji.



Diagramy ERD (Entity Relationship Diagram)

Do tworzenia modelu graficznego schematu bazy danych wykorzystywane są diagramy związków encji, z których najpopularniejsze są diagramy **ERD (ang. Entity Relationship Diagram)**. Pozwalają one na:

1. **Modelowanie struktur danych** oraz związków zachodzących między tymi strukturami
2. **Prawie bezpośrednie przekształcenie** diagramu w schemat relacyjny
3. **Analizę struktury** bazy danych na wysokim poziomie abstrakcji
4. **Dokumentację** tworzonego systemu baz danych

Na diagramy ERD składają się trzy rodzaje elementów:

- ✓ zbiory encji,
- ✓ atrybuty encji,
- ✓ związki zachodzące między encjami.



1. Zbiory encji (Entities)

Encja to reprezentacja obiektu przechowywanego w bazie danych. Graficzną reprezentacją encji jest najczęściej prostokąt.

Przykład:



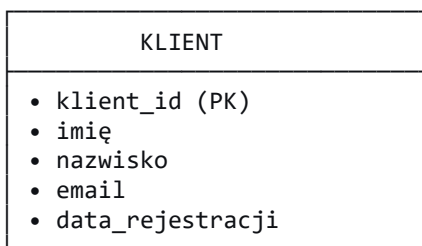
2. Atrybuty encji (Attributes)

Atrybut opisuje encję. Może on być liczbą, tekstem lub wartością logiczną. W relacyjnym modelu baz danych atrybut jest reprezentowany przez kolumnę tabeli.

Reprezentacje atrybutów:

- W notacji Chena: owal połączony z encją
- W notacji "pudełkowej": lista wewnątrz prostokąta encji

Przykład encji Klient z atrybutami:



3. Związki między encjami (Relationships)

Związek to powiązanie między dwoma zbiorami encji. Każdy związek ma dwa końce, do których są przypisane:

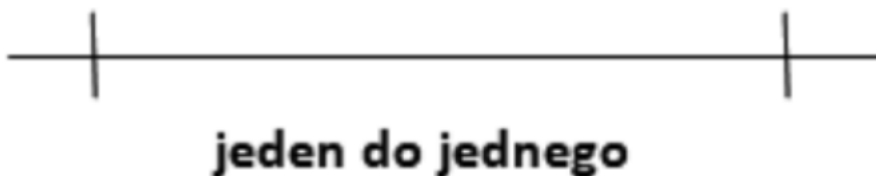


a) Stopień związku (Cardinality)

Określa, jakiego typu związek zachodzi między encjami:



1. Jeden do jednego (1:1)

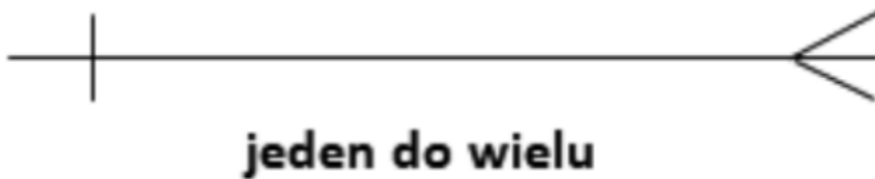


Przykład: Pracownik ↔ Samochód służbowy

2.



Jeden do wielu (1:N)

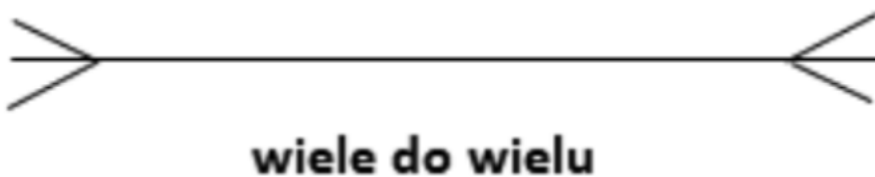


Przykład: Klient ↔ Zamówienia

3.



Wiele do wielu (N:M)



Przykład: Student ↔ Kursy

b)



Opcjonalność związku (Optionality)

Określa, czy związek jest opcjonalny, czy wymagany:

•



Wymagany (mandatory):



związek wymagany

•



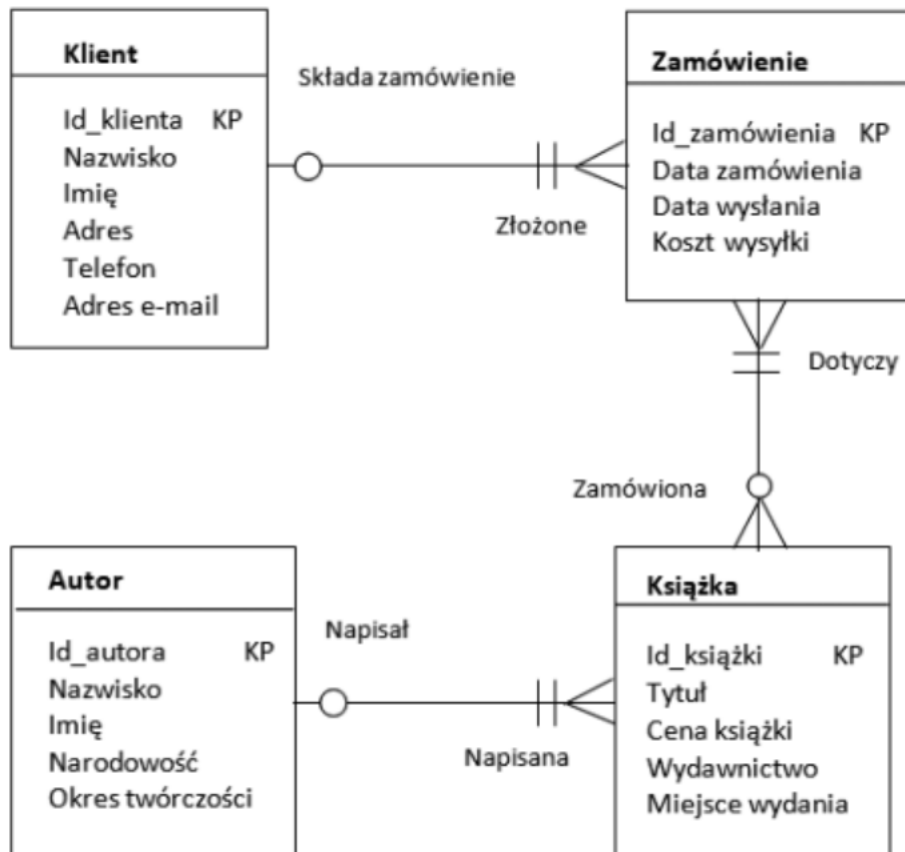
Opcjonalny (optional):



związek opcjonalny

Diagramy ERD spotyka się w wielu różnych notacjach, na przykład: Martina, Bachmana, Chena, IDEF1X

Prosty przykład diagramu ERD w notacji Martina:



Transformacja modelu konceptualnego na model relacyjny

1. Podstawowe Pojęcia



Model konceptualny – to abstrakcyjny, wysokopoziomowy opis danych, zwykle tworzony w ERD (Entity-Relationship Diagram).

- Mówimy o **encjach** (np. Osoba, Paszport)
- Ich **atrybutach** (np. imię, nazwisko, numer_pasportu)
- Związkach między encjami (1:1, 1:N, N:M)



Model relacyjny – to konkretny sposób zapisania tych danych w tabelach w relacyjnej bazie danych.

- Tworzymy **tabele**, **kolumny**, **klucze główne** i **obce**
- Zachowujemy struktury z modelu konceptualnego

Transformacja = proces przekształcania **ERD** w **tabele relacyjne**.

2. Podstawowe Zasady Transformacji



Encja → Tabela

Każda encja staje się tabelą w bazie danych.



Model konceptualny:

ENCJA: Student

```
|  
├─ Atrybuty:  
|   ├─ id_studenta (PK)  
|   ├─ imie  
|   ├─ nazwisko  
|   ├─ data_urodzenia  
|   └─ email
```



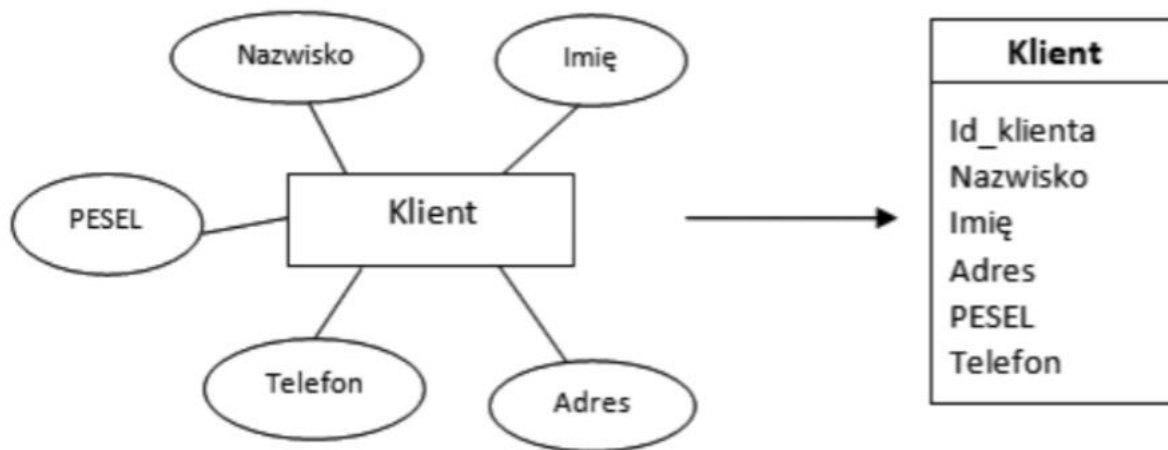
Model relacyjny:

```
CREATE TABLE Student (  
    id_studenta INT PRIMARY KEY,  
    imie VARCHAR(50) NOT NULL,  
    nazwisko VARCHAR(50) NOT NULL,  
    data_urodzenia DATE,  
    email VARCHAR(100) UNIQUE  
);
```

Do opisu każdego zbioru podobnych encji stosuje się oddzielną tabelę. Jednej encji odpowiada jeden wiersz. Atrybutowi odpowiada kolumna. Dla każdego atrybutu określa się typ informacji.

Atrybuty → Kolumny

Atrybuty encji stają się kolumnami tabeli.



3. Transformacja Relacji

Relacja 1:1 (jeden do jednego)

Przykład: Student ↔ Legitymacja

Model koncepcyjny:

Student (1) — posiada — (1) Legitymacja

Model relacyjny - Opcja A (klucz obcy w jednej tabeli):

```
CREATE TABLE Student (
    id_studenta INT PRIMARY KEY,
    imie VARCHAR(50),
    nazwisko VARCHAR(50)
);
```

```
CREATE TABLE Legitymacja (
    id_legitymacji INT PRIMARY KEY,
    numer VARCHAR(20) UNIQUE,
    data_wydania DATE,
    id_studenta INT UNIQUE, -- Klucz obcy + UNIQUE
    FOREIGN KEY (id_studenta) REFERENCES Student(id_studenta)
);
```

Relacja 1:N (jeden do wielu)

Przykład: Wykładowca (1) ↔ Przedmioty (N)

Model koncepcyjny:

Wykładowca (1) — prowadzi — (N) Przedmiot

```
CREATE TABLE Wykladowca (  
    id_wykladowcy INT PRIMARY KEY,  
    imie VARCHAR(50),  
    nazwisko VARCHAR(50),  
    tytul_naukowy VARCHAR(50)  
);  
  
CREATE TABLE Przedmiot (  
    id_przedmiotu INT PRIMARY KEY,  
    nazwa VARCHAR(100),  
    ects INT,  
    id_wykladowcy INT,  
    FOREIGN KEY (id_wykladowcy) REFERENCES Wykladowca(id_wykladowcy)  
);
```

Relacja M:N (wiele do wielu)

Przykład: Student (N) ↔ Przedmiot (M)

Model konceptualny:

Student (N) — zapisuje się na — (M) Przedmiot

Model relacyjny (tabela asocjacyjna):

```
CREATE TABLE Student (  
    id_studenta INT PRIMARY KEY,  
    imie VARCHAR(50),  
    nazwisko VARCHAR(50)  
);  
  
CREATE TABLE Przedmiot (  
    id_przedmiotu INT PRIMARY KEY,  
    nazwa VARCHAR(100),  
    ects INT  
);  
  
-- TABELA ASOCJACYJNA lub pośrednia  
CREATE TABLE Zapis_na_przedmiot (  
    id_zapisu INT PRIMARY KEY,  
    id_studenta INT,  
    id_przedmiotu INT,  
    data_zapisu DATE,  
    ocena DECIMAL(3,2),  
    FOREIGN KEY (id_studenta) REFERENCES Student(id_studenta),  
    FOREIGN KEY (id_przedmiotu) REFERENCES Przedmiot(id_przedmiotu),  
    UNIQUE(id_studenta, id_przedmiotu)  
);
```

Do tej lekcji włącznie:

Normalizacja bazy danych. Wybór struktur i określenie zasad dostępu do bazy danych

Lekcja

Temat: Normalizacja bazy danych. Wybór struktur i określenie zasad dostępu do bazy danych



Normalizacja tabel

Normalizację stosuje się, aby sprawdzić, czy zaprojektowane tabele mają prawidłową strukturę. Kiedy wstępny projekt tabel jest gotowy, rozpoczynamy normalizację. Pozwala określić, czy założenia projektu zostały przydzielone do właściwych tabel. Natomiast nie da odpowiedzi na pytanie, czy projekt bazy danych jest prawidłowy.



Stosowane są cztery reguły normalizacji, ale w większości projektów baz danych wystarczy sprawdzić trzy pierwsze.

Dla każdej z nich stosowane są określenia:

- ✓ pierwsza postać normalna (I PN),
- ✓ druga postać normalna (II PN),
- ✓ trzecia postać normalna (III PN).

Pierwsza postać normalna



Tabela jest w pierwszej postaci normalnej (I PN), gdy każdy wiersz w tabeli przechowuje informacje o pojedynczym obiekcie, a każde pole tabeli zawiera informację elementarną (atomową).

Przykłady:



NIEWŁAŚCIWE (nieatomowe wartości):

Tabela Klienci:

ID	Imię	Telefony
----	------	----------

1	Jan	123-456-789, 987-654-321
2	Anna	555-111-222

Problem: Kolumna "Telefony" zawiera 2 numery w jednej komórce, rozdzielone przecinkiem.



WŁAŚCIWE (wartości atomowe):

Rozwiązanie 1: Dla telefonów - osobne rekordy

ID	ID_Klienta	Telefon
1	1	123-456-789
2	1	987-654-321
3	2	555-111-222



Jak sprawdzić, czy wartość jest atomowa?

Zadaj pytanie: "Czy kiedykolwiek będę potrzebował tylko części tej wartości?"

- Jeśli odpowiedź brzmi "**tak**" → prawdopodobnie nie jest atomowa
- Jeśli odpowiedź brzmi "**nie**, zawsze używam całej wartości" → prawdopodobnie jest atomowa

Druga postać normalna



Tabela jest w drugiej postaci normalnej (II PN), jeżeli jest w pierwszej postaci normalnej (I PN) oraz każde z pól niewchodzących w skład klucza podstawowego zależy od całego klucza, a nie od jego części.



PRZYKŁAD PRZED 2NF (spełnia 1NF, ale nie 2NF):

Wyobraźmy się sklep internetowy. Jedna tabela przechowuje **pozycje zamówień**:

Tabela Pozycje_Zamowienia:

ID_Zamowienia	ID_Produktu	Ilosc	Nazwa_Produktu	Cena_Produktu	Kategoria
100	5	2	Laptop Gaming	4500	Elektronika
100	8	1	Mysz bezprzewodowa	200	Akcesoria
101	5	1	Laptop Gaming	4500	Elektronika
102	8	3	Mysz bezprzewodowa	200	Akcesoria

- **Klucz główny (złożony):** (ID_Zamowienia, ID_Produktu)
 - o Ta kombinacja jednoznacznie identyfikuje każdą pozycję (w zamówieniu 100 może być tylko jeden produkt 5).
- **Koluby:**
 - o Ilość zależy od **całego klucza**. Aby wiedzieć, ile sztuk produktu 5 zamówiono, muszę znać **zarówno numer zamówienia (100), jak i produkt (5)**.
 - o Nazwa_Produktu, Cena_Produktu, Kategoria zależą **tylko od części klucza** (ID_Produktu). Nazwa "Laptop Gaming" jest taka sama, niezależnie od tego, w którym zamówieniu (100 czy 101) się pojawia.



PROBLEMY (anomalie):

1. **Niespójność przy aktualizacji:** Jeśli zmienię nazwę produktu z "Laptop Gaming" na "Laptop Extreme", muszę zaktualizować **wszystkie wiersze** gdzie ID_Produktu=5. Jeśli pominę jeden, dane stają się niespójne.
2. **Redundancja (powtarzanie):** Dane o produkcie (nazwa, cena, kategoria) powtarzają się przy każdym jego zamówieniu.
3. **Wstawianie:** Nie mogę dodać nowego produktu do bazy, dopóki nie zostanie on zamówiony (bo ID_Zamowienia jest częścią klucza i nie może być NULL).

4. **Usuwanie:** Jeśli usunę ostatnie zamówienie zawierające produkt 5 (np. zamówienie 101), tracę informację o tym produkcie (jego nazwę, cenę) z bazy.



ROZWIĄZANIE PO NORMALIZACJI DO 2NF:

Dzielimy tabelę na dwie, usuwając zależności częściowe. Atrybuty zależne tylko od ID_Projektu przenosimy do nowej tabeli.

Tabela 1: Pozycje_Zamowienia (przechowuje fakty o konkretnym zamówieniu)

ID_Zamowienia	ID_Projektu	Ilosc
100	5	2
100	8	1
101	5	1
102	8	3

- Klucz główny: (ID_Zamowienia, ID_Projektu)
- Jedyna pozakluczowa kolumna Ilość zależy teraz od **całego klucza**.

Tabela 2: Produkty (przechowuje stałe dane katalogowe produktów)

ID_Projektu	Nazwa_Projektu	Cena_Projektu	Kategoria
5	Laptop Gaming	4500 zł	Elektronika
8	Mysz bezprzewodowa	200 zł	Akcesoria

- Klucz główny: ID_Projektu
- Wszystkie kolumny zależą od całego tego klucza.



Jak teraz wyglądają problemy?

1. **Aktualizacja:** Aby zmienić nazwę produktu, aktualizuję **jeden wiersz** w tabeli Produkty.
2. **Redundancja:** Dane o produkcie są przechowywane tylko raz.
3. **Wstawianie:** Mogę dodać nowy produkt do tabeli Produkty, nawet jeśli nikt go jeszcze nie zamówił.
4. **Usuwanie:** Usunięcie zamówienia z tabeli Pozycje_Zamowienia **nie usuwa informacji o produkcie** z katalogu.

Podsumowanie 2NF w praktyce:



Jeśli twoja tabela ma **klucz pojedynczy** (np. ID_Zamowienia), to automatycznie spełnia 2NF (nie ma od czego być "częściowo zależna").

2NF ma sens głównie dla tabel z kluczami złożonymi, które przechowują dane z różnych "poziomów" lub encji (jak pozycja zamówienia + dane produktu).

Trzecia postać normalna



Tabela jest w trzeciej postaci normalnej (III PN), jeżeli jest w pierwszej i w drugiej postaci normalnej oraz każde z pól niewchodzących w skład klucza podstawowego niesie informację bezpośrednio o kluczu i nie odnosi się do żadnego innego pola.

Przykład przed i po normalizacji:

Przed normalizacją:

Tabela Zamowienia:

ID_Zamowienia	Klient	Produkt	Cena_Produktu	Kategoria_Produktu
1	Kowal	Laptop	3000	Elektronika
2	Nowak	Laptop	3000	Elektronika

Problem: Powtarzające się dane o produkcie

Po normalizacji (3NF):

Tabela Zamowienia:

ID_Zamowienia	ID_Klienta	ID_Produktu
1	101	500
2	102	500

Tabela Klienci:

ID_Klienta	Nazwisko
101	Kowalski
102	Nowak

Tabela Produkty:

ID_Produktu	Nazwa	Cena	ID_Kategorii
500	Laptop	4000	10

Tabela Kategorie:

ID_Kategorii	Nazwa
10	Elektronika

Zalety normalizacji:

- Mniejsza redundancja danych
- Lepsza integralność danych
- Łatwiejsze utrzymanie i modyfikacje
- Spójność danych

Wady:

- Złożone zapytania z wieloma JOIN
- Możliwy spadek wydajności przy bardzo rozdrobnionych tabelach
- Czasami stosuje się **denormalizację** celowo dla poprawy wydajności

Tego już nie będzie:

Wybór struktur i określenie zasad dostępu do bazy danych

Wybór struktur i określenie zasad dostępu do bazy danych



Od wersji MySQL 5.5 (wydanej w 2010) domyślnym i rekomendowanym silnikiem jest InnoDB. W 99% przypadków to właściwy wybór.



Zalety:

1. **Transakcje ACID** - gwarancja spójności danych
2. **Klucze obce** - integralność relacyjna
3. **Row-level locking** - lepsza współbieżność (**blokowanie tylko konkretne wiersze (rekordy)** w tabeli, a nie całą tabelę). **Współbieżność (concurrency)** to zdolność systemu do obsługi wielu użytkowników jednocześnie.

Im mniej danych jest blokowanych, tym:więcej zapytań może działać równolegle. Mniej operacji czeka na zwolnienie blokady system działa szybciej pod obciążeniem

4. **Crash recovery** - odtwarza się po awarii. Odtwarza zatwierdzone transakcje i cofa niezakończone (Jeśli jakaś transakcja: była rozpoczęta, ale NIE miała COMMIT,i nastąpił crash,to InnoDB ją cofa), przywracając bazę do spójnego stanu.
5. **MVCC** (Multi-Version Concurrency Control) - jednoczesne odczyty i zapisy. Zamiast blokować dane, baza przechowuje wiele wersji tego samego wiersza.
6. **Optymalizacje dla nowoczesnego hardware** (SSD, duża pamięć RAM)
7. Kompresja danych
8. Obsługa dużych obiektów (BLOB/TEXT)



```
CREATE TABLE users (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100),  
  email VARCHAR(100) UNIQUE  
) ENGINE=InnoDB;
```



Wady:

1. Większy rozmiar plików niż MyISAM
2. Brak FULLTEXT indeksów (ale od MySQL 5.6+ są!)
3. Wolniejsze COUNT(*) bez WHERE



2. MyISAM - STARSZY, CORAZ RZADZIEJ UŻYWANY

```
CREATE TABLE logs (  
  id INT PRIMARY KEY,  
  log_date DATE,  
  message TEXT,  
  FULLTEXT(message)  
) ENGINE=MyISAM
```

Zalety:

- Szybsze odczyty dla tabel tylko do odczytu
- Mniejszy rozmiar dysku
- Pełnotekstowe indeksy (w starszych wersjach)
- Dobry do tabel tymczasowych

Wady:

- **Brak transakcji**
- **Table-level locking** (blokada całej tabeli przy zapisie)
- Brak kluczy obcych
- Podatny na uszkodzenie przy awarii
- Brak crash recovery

Użyj gdy: Tabele tylko do odczytu (archiwa), logi (gdy nie potrzebujesz transakcji), w systemach reportingowych.

3. MEMORY (HEAP) - TABELA W PAMIĘCI RAM

```
CREATE TABLE session_data (  
    session_id VARCHAR(32) PRIMARY KEY,  
    user_id INT,  
    data TEXT,  
    created TIMESTAMP  
) ENGINE=MEMORY;
```

Zalety:

- **Ekstremalnie szybki dostęp**
- Niski overhead
- Hash indeksy domyślnie

Wady:

- **Dane tracone po restarcie MySQL**
- Obsługuje tylko typy o stałej długości
- Ograniczony rozmiar (zależy od max_heap_table_size)
- Table-level locking

Użyj gdy: Cache, dane sesji, tabele tymczasowe, szybkie lookup tables.

4. ARCHIVE - KOMPRESJA DANYCH

```
CREATE TABLE audit_logs (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    action VARCHAR(50),  
    details TEXT,  
    created DATETIME  
) ENGINE=ARCHIVE;
```

Zalety:

- **Bardzo dobra kompresja** (do 90%)
- Mały rozmiar na dysku
- Szybkie INSERT (kompresja on-the-fly)

Wady:

- **Tylko INSERT i SELECT** (brak UPDATE/DELETE)
- Brak indeksów (poza PRIMARY KEY)
- Brak transakcji

Użyj gdy: Archiwizacja danych (logi, historyczne rekordy), backup historyczny.

5. CSV - DANE W PLIKU CSV

```
CREATE TABLE import_data (  
  id INT,  
  name VARCHAR(100),  
  value DECIMAL(10,2)  
) ENGINE=CSV;
```

Zalety:

- Prosty import/eksport do Excel
- Czytelny format
- Łatwa integracja z innymi narzędziami

Wady:

- Brak indeksów
- Brak typów danych BLOB/TEXT
- Wszystko jako tekst

Użyj gdy: Import/export danych, integracja z zewnętrznymi systemami.

6. BLACKHOLE - DANE ZNIKAJĄ

```
CREATE TABLE replication_filter (  
  id INT,  
  data VARCHAR(255)  
) ENGINE=BLACKHOLE;
```

Zalety:

- Dane "znikają" po INSERT
- Niski overhead
- Przydatny do replikacji

Wady:

- Brak przechowywania danych
- Tylko struktura tabeli

Użyj gdy: Filtrowanie replikacji, testowanie wydajności, logowanie bez zapisu.

7. FEDERATED - DANE Z ZEWNĘTRZNEJ BAZY

```
CREATE TABLE remote_table (  
  id INT PRIMARY KEY,  
  name VARCHAR(100)
```

```
) ENGINE=FEDERATED  
CONNECTION='mysql://user:pass@remote_host:3306/db/table';
```

Zalety:

- Dostęp do zdalnych danych jak do lokalnych
- Transparentność lokalizacji

Wady:

- Wolniejsze (dostęp przez sieć)
- Problemy z łącznością
- Ograniczone wsparcie

Użyj gdy: Integracja rozproszonych baz, agregacja danych.

TABELA PORÓWNAWCZA

Silnik	Transakcje	Klucze obce	Locking	MVCC	Crash Safe	Pamięć	Użycie
InnoDB	☑	☑	Row-level	☑	☑	Duża	Domyślny
MyISAM	✗	✗	Table-level	✗	✗	Mała	Tylko odczyt
MEMORY	✗	✗	Table-level	✗	✗	RAM	Cache
ARCHIVE	✗	✗	Row-level	✗	✗	Bardzo mała	Archiwa
CSV	✗	✗	Table-level	✗	✗	Średnia	Import/Export
BLACKHOLE	✗	✗	Table-level	✗	☑	Brak	Replikacja

PRAKTYCZNE WSKAZÓWKI WYBORU

Wybierz InnoDB gdy:

- Tworzysz nową aplikację
- Potrzebujesz transakcji
- Masz relacje między tabelami
- Chcesz bezpieczeństwa danych
- Masz równoczesnych wielu użytkowników

Wybierz MyISAM gdy:

- Masz tabelę tylko do odczytu
- Masz stare aplikacje, które go wymagają
- Potrzebujesz FULLTEXT w MySQL < 5.6
- Masz ograniczony dysk

RODZAJE INDEKSÓW W MYSQL

Podstawowe typy:

```
-- 1. PRIMARY KEY (domyślnie clustered index w InnoDB)
CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT, -- Automatycznie indeks
  username VARCHAR(50)
);

-- 2. UNIQUE INDEX (gwarantuje unikalność)
CREATE UNIQUE INDEX idx_email ON users(email);

-- 3. INDEX (zwykły indeks)
CREATE INDEX idx_last_name ON customers(last_name);

-- 4. FULLTEXT (wyszukiwanie pełnotekstowe)
CREATE FULLTEXT INDEX idx_content ON articles(body);

-- 5. SPATIAL (dla danych geograficznych)
CREATE SPATIAL INDEX idx_location ON places(coordinates);
```

Struktury indeksów:

- **B-Tree** (domyślnie) - zakresy, sortowanie, przedrostki
- **Hash** (tylko MEMORY/NDB) - tylko równości (=)
- **R-Tree** (SPATIAL) - dane przestrzenne

STRATEGIE TWORZENIA INDEKSÓW

Kiedy tworzyć indeksy?

Twórz indeksy dla:

```
-- WHERE conditions
CREATE INDEX idx_status ON orders(status);
-- SELECT * FROM orders WHERE status = 'shipped'

-- JOIN columns
CREATE INDEX idx_customer_id ON orders(customer_id);
-- SELECT * FROM customers JOIN orders ON customers.id = orders.customer_id

-- ORDER BY / GROUP BY
CREATE INDEX idx_order_date ON orders(order_date);
-- SELECT * FROM orders ORDER BY order_date DESC

-- Combination
CREATE INDEX idx_status_date ON orders(status, order_date);
-- SELECT * FROM orders WHERE status = 'pending' ORDER BY order_date
```

Kiedy NIE tworzyć indeksów?

- Tabele małe (< 1000 wierszy)
- Kolumny często modyfikowane (INSERT/UPDATE/DELETE)
- Kolumny z niską selektywnością (np. "gender" z wartościami M/F)
- Kolumny typu BLOB/TEXT (chyba że prefiksy)

3. INDEKSY ZŁOŻONE (COMPOSITE) - NAJWAŻNIEJSZE!

Zasada kolejności kolumn:

```
-- DOBRZE: (last_name, first_name)
CREATE INDEX idx_name ON employees(last_name, first_name);

-- Zapytania wykorzystujące indeks:
SELECT * FROM employees WHERE last_name = 'Kowalski';
SELECT * FROM employees WHERE last_name = 'Kowalski' AND first_name = 'Jan';
SELECT * FROM employees WHERE last_name LIKE 'Kow%';

-- NIE wykorzysta indeksu dla:
SELECT * FROM employees WHERE first_name = 'Jan'; -- Brak last_name!

-- RÓWNIEŻ DOBRE: (status, order_date, customer_id)
CREATE INDEX idx_order_filter ON orders(status, order_date, customer_id);
```

Kardynalność - kolejność kolumn w indeksie złożonym:

```
-- ZŁE: (gender, last_name) - gender ma niską kardynalność
CREATE INDEX idx_gender_name ON users(gender, last_name);

-- DOBRE: (last_name, gender) - last_name ma wysoką kardynalność
CREATE INDEX idx_name_gender ON users(last_name, gender);

-- REGUŁA: Kolumny z wyższą kardynalnością (więcej unikalnych wartości) NA PRZODZIE
```

4. INDEKSY POKRYWAJĄCE (COVERING INDEXES)

Indeks zawiera WSZYSTKIE kolumny potrzebne w zapytaniu:

```
-- Tabela: orders(id, customer_id, amount, status, order_date)

-- ZAPYTANIE 1: Potrzebuje skanowania tabeli
SELECT customer_id, amount, status
FROM orders
WHERE order_date > '2024-01-01';

-- ROZWIĄZANIE: Covering index
CREATE INDEX idx_covering ON orders(order_date, customer_id, amount, status);
-- Teraz MySQL pobierze dane TYLKO z indeksu (fast!)

-- ZAPYTANIE 2:
SELECT COUNT(*) FROM orders WHERE status = 'completed';
```

```
-- Covering index:
CREATE INDEX idx_status_covering ON orders(status);
```

5. ANALIZA WYKORZYSTANIA INDEKSÓW

EXPLAIN - najważniejsze narzędzie:

```
EXPLAIN FORMAT=JSON
SELECT o.*, c.name
FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE o.status = 'shipped'
AND o.order_date > '2024-01-01'
ORDER BY o.order_date DESC;
```

-- Kluczowe pola w wyniku EXPLAIN:
-- type: ALL (full scan) | index | range | ref | eq_ref | const
-- key: który indeks został użyty
-- rows: szacowana liczba przeszukiwanych wierszy
-- Extra: Using index (covering), Using filesort (problem!), Using temporary

Model warstwowy bezpieczeństwa:

Warstwa 5: Monitoring i audyt	← Kto, co, kiedy zrobił
Warstwa 4: Szyfrowanie danych	← DANE W SPOCZYNKU
Warstwa 3: Kontrola dostępu	← UPRAWNIENIA
Warstwa 2: Uwierzytelnianie	← LOGOWANIE
Warstwa 1: Sieć i firewall	← DOSTĘP SIECIOWY

Tworzenie ról (MySQL 8.0+):

```
-- 1. Definiowanie ról
CREATE ROLE 'app_readonly';
CREATE ROLE 'app_editor';
CREATE ROLE 'app_admin';
CREATE ROLE 'reporting_user';
```

```
-- 2. Przypisywanie uprawnień do ról
GRANT SELECT ON ecommerce.* TO 'app_readonly';
GRANT SELECT, INSERT, UPDATE ON ecommerce.orders TO 'app_editor';
GRANT SELECT, INSERT, UPDATE ON ecommerce.customers TO 'app_editor';
GRANT ALL PRIVILEGES ON ecommerce.* TO 'app_admin';
GRANT SELECT ON ecommerce.sales_view TO 'reporting_user';

-- 3. Tworzenie użytkowników
CREATE USER 'api_user'@'10.0.0%' IDENTIFIED BY 'StrongPass123!';
CREATE USER 'analyst_john'@'localhost' IDENTIFIED WITH caching_sha2_password BY 'AnalystPass!2024';
CREATE USER 'backup_user'@'localhost' IDENTIFIED BY 'BackupPass456$';

-- 4. Przypisywanie ról użytkownikom
GRANT 'app_editor' TO 'api_user'@'10.0.0%';
GRANT 'reporting_user' TO 'analyst_john'@'localhost';
GRANT SELECT, RELOAD, LOCK TABLES, REPLICATION CLIENT ON *.* TO 'backup_user'@'localhost';

-- 5. Aktywacja ról (MySQL wymaga jawnej aktywacji)
SET DEFAULT ROLE ALL TO 'api_user'@'10.0.0%';
```

To **kopia zapasowa** Twoich danych. Jak zdjęcie całej bazy w danym momencie.

Bo błędy się zdarzają:

- Ktoś usunie ważne dane
- Zepsuje się dysk serwera
- Atak hakerski (ransomware)
- Pożar/powódź w serwerowni

Bez backupu = TRACISZ DANE NA ZAWSZE

ZASADA 3-2-1-1-0 (NAJPROŚCIEJ)

Zapamiętaj te liczby:

3 - Trzy kopie tego samego

Przykład:

1. Na serwerze produkcyjnym
2. Na dysku lokalnym
3. W chmurze/innej lokalizacji

2 - Dwa różne nośniki

Przykład:

- Dysk twardy w serwerze
- Zewnętrzny dysk/taśma LTO

1 - Jedna kopia poza firmą

Backup w innym mieście/budynku.

Gdy spłonie serwerownia, backup jest bezpieczny.

1 - Jedna kopia OFFLINE

Dysk **niepodłączony do sieci**.

Ochrona przed ransomware - haker nie zaszyfruje odłączonego dysku!

0 - Zero błędów

Backup musi być sprawdzony i działający.

JAK CZĘSTO ROBIĆ BACKUP?

Dla małej firmy/serwera:

Codziennie 2:00 w nocy:
(cała baza)

← PEŁNY backup

Co 4 godziny:
(tylko zmiany)

← PRZYROSTOWY

Dla większego systemu:

Niedziela 2:00 → PEŁNY backup (całość)

Pon-Pt 1:00 → PRZYROSTOWY (tylko zmiany)

Co 15 minut → LOGI zmian (najmniejsze straty)

3 RODZAJE BACKUPÓW:

1. PEŁNY (FULL)

- **Co to?** Cała baza
- **Kiedy?** Raz w tygodniu
- **Plusy:** łatwe przywracanie
- **Minusy:** Dużo miejsca, długo trwa

2. PRZYROSTOWY (INCREMENTAL)

- **Co to?** Tylko to, co się zmieniło od ostatniego backupu

- **Kiedy?** Codziennie
- **Plusy:** Mało miejsca, szybko
- **Minusy:** Przywracanie wolniejsze (trzeba połączyć kilka backupów)

3. LOGI (BINLOG)

- **Co to?** Zapis każdej zmiany w bazie (jak rejestrator)
- **Kiedy?** Co 15-60 minut
- **Plusy:** Przywrócisz do dowolnej minuty!
- **Minusy:** Trzeba łączyć z backupem pełnym

PROSTA ANALOGIA:

Wyobraź sobie bibliotekę:

1. **Pełny backup** = Zrobienie ksero **całej biblioteki** (raz w tygodniu)
2. **Przyrostowy** = Ksero tylko **nowych książek** (codziennie)
3. **Logi** = Zapis **kto i kiedy wypożyczył** (co godzinę)

Co się stanie, gdy pożar zniszczy bibliotekę?

1. Odtwarzasz z ksero całej biblioteki (pełny backup)
2. Dodajesz nowe książki (przyrostowe)
3. Sprawdzasz wypożyczenia (logi)

KROK 1: Określ co backupować

- Tylko baza danych? ✓
- Konfiguracja serwera? ✓
- Pliki aplikacji? ✓
- Logi systemowe? (opcjonalnie)

KROK 2: Ustal częstotliwość

- **RPO** (Recovery Point Objective) = Ile danych możesz stracić?
 - Bank: **0 minut** (każda transakcja ważna)
 - Blog: **24 godziny** (można stracić komentarze z dnia)

KROK 3: Wybierz gdzie przechowywać

MUST HAVE:

- Dysk lokalny (szybki dostęp)
- Zewnętrzny dysk (offline)
- Chmura (offsite)

NIGDY NIE:

- Tylko jeden dysk
- Tylko w tym samym budynku
- Tylko online

KROK 4: Automatyzuj

```
# Przykładowy skrypt
#!/bin/bash
# Codziennie 2:00 wykonaj backup
mysqldump -u root -pHaslo123 baza_firmy > /backup/baza_$(date +%Y%m%d).sql
# Wyślij na zewnętrzny dysk
cp /backup/baza_*.sql /mnt/external_drive/
# Wyślij do chmury
rclone copy /backup/ backup_cloud:
```

KROK 5: TESTUJ!

Najważniejsza zasada:

Backup bez testu = brak backupu

Lekcja

Temat: Systemy zarządzania bazą danych



System zarządzania bazą danych (w skrócie **SZBD**; po angielsku: *Database Management System* – **DBMS**) to specjalne oprogramowanie, które służy do tworzenia, zarządzania i przetwarzania danych w bazach danych.



Główne zadania SZBD

Zadanie	W praktyce
Definiowanie danych	Tworzenie struktury bazy (tabele, kolumny, relacje)
Manipulacja danymi	Dodawanie, usuwanie, modyfikowanie i wyszukiwanie rekordów
Kontrola współbieżności	Wielu użytkowników może pracować jednocześnie bez kolizji
Bezpieczeństwo i autoryzacja	Hasła, role, szyfrowanie, logi kto i co zmienił
Integralność danych	Zapobieganie „bzdurnym” danym (np. data urodzenia w przyszłości)
Backup i odzyskiwanie	Regularne kopie + możliwość przywrócenia bazy po awarii
Optymalizacja wydajności	Indeksy, partycjonowanie, cache, zapytania optymalizowane
Transakcje (ACID)	Gwarancja, że operacje albo wykonają się w całości, albo wcale ⁱ



Najpopularniejsze rodzaje SZBD

Typ	Przykłady	Kiedy się używa
Relacyjny (RDBMS)	MySQL, PostgreSQL, Oracle, SQL Server, MariaDB, SQLite	Finanse, e-commerce, systemy firmowe, aplikacje webowe
NoSQL dokumentowy	MongoDB, CouchDB	Aplikacje z dużą ilością niestrukturyzowanych danych (JSON)
NoSQL kolumnowy	Cassandra, HBase	Bardzo duże zbiory danych, analityka, Big Data
NoSQL grafowy	Neo4j, Amazon Neptune	Sieci społecznościowe, rekomendacje, wykrywanie oszustw
Klucz-wartość	Redis, DynamoDB	Cache, sesje użytkowników, liczniki
NewSQL	CockroachDB, TiDB, YugabyteDB	Połączenie zalet RDBMS i skalowalności NoSQL

Najważniejsze cechy dobrego SZBD (2025/2026)

- Wsparcie dla **chmury** (AWS RDS, Azure SQL, Google Cloud SQL, Supabase, Neon, PlanetScale)
- Wbudowane **AI/ML** do automatycznego tuningu i wykrywania anomalii (np. Oracle Autonomous, Azure SQL Hyperscale)
- **Zero-downtime** – możliwość aktualizacji bez zatrzymywania bazy
- Łatwa **replikacja** i **sharding** (rozbijanie bazy na wiele serwerów)

Architektura systemu baz danych to sposób, w jaki zorganizowane są komponenty oprogramowania współpracujące z danymi. Najczęściej mówiąc o architekturze baz danych, mamy na myśli **architekturę ANSI/SPARC** (podział na trzy poziomy) lub **topologię komunikacji** (rodzaje systemów ze względu na to, gdzie znajdują się dane i jak są przetwarzane).



1. Architektura trójpoziomowa (ANSI/SPARC)

To koncepcyjny model pokazujący, jak użytkownik "widzi" dane w stosunku do tego, jak są fizycznie przechowywane. **Dzieli się na trzy warstwy:**



A. Poziom zewnętrzny (Widoki użytkowników, ang. External level)

To, co widzi konkretny użytkownik lub aplikacja.

- **Opis:** Każdy użytkownik ma dostęp tylko do wycinka danych, który jest mu potrzebny. Reszta bazy jest dla niego niewidoczna.
- **Przykład:** Pracownik kadr widzi dane osobowe i pensję, ale nie widzi danych logistycznych. Kurier widzi adresy dostaw, ale nie widzi pensji kolegów. Kasjer w sklepie widzi ceny, ale nie widzi marży.



B. Poziom koncepcyjny (Logiczny, ang. Conceptual level)

To "spis treści" całej bazy danych.

- **Opis:** Opisuje, jakie dane są przechowywane i jakie są między nimi powiązania. Nie przejmuje się tym, jak fizycznie wyglądają te dane na dysku.
- **Przykład:** Definiujemy, że w systemie są "Klienci" (mający ID, Imię, Email) i "Zamówienia" (mające ID, Datę, ID_Klienta). To właśnie ten poziom.



C. Poziom wewnętrzny (Fizyczny, ang. Internal level)

To fizyczne składowanie danych na dysku twardym.

- **Opis:** Określa, jak dane są zapisywane w postaci plików, bajtów i sektorów, jakie indeksy przyspieszają wyszukiwanie, czy dane są skompresowane itp.
- **Przykład:** Na tym poziomie decydujemy, że tabela "Klienci" będzie zapisana w pliku `klienci.myd` na partycji D:, a dla kolumny "Email" zostanie stworzony specjalny indeks, by szybko wyszukiwać maile.



2. Architektura ze względu na topologię (Rodzaje SZBD)

To, jak system jest rozłożony w sieci i gdzie odbywa się przetwarzanie danych.



A. Architektura centralna (monolityczna)

Baza danych i program, który z niej korzysta, działają na tym samym komputerze.

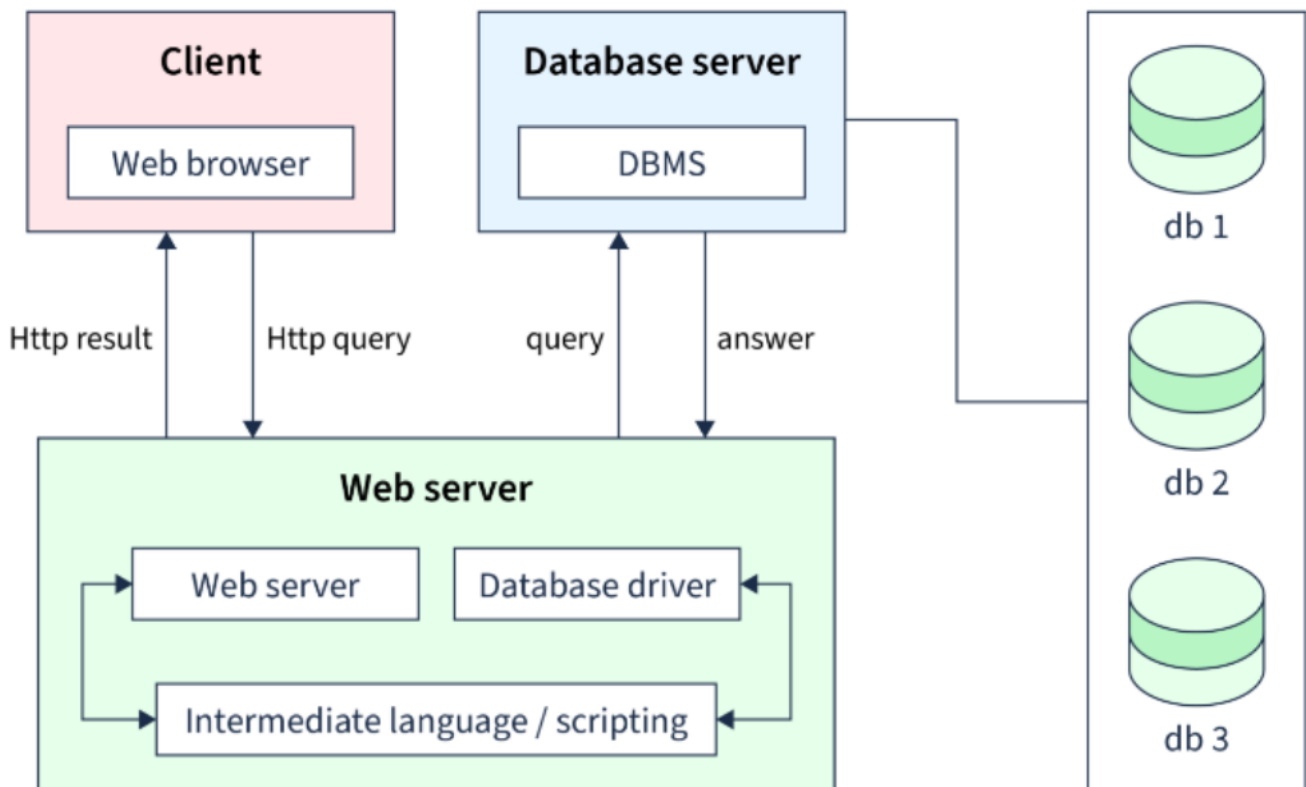
- **Działanie:** Aplikacja łączy się bezpośrednio z silnikiem bazy danych na lokalnym dysku.
- **Przykład:** Aplikacja księgowa na laptopie księgowego korzystająca z SQLite. Mała, przenośna, ale nieprzeznaczona do pracy grupowej.



B. Architektura dwuwarstwowa (klient-serwer)

To najpopularniejszy model klasycznych baz danych.

- **Działanie:**
 - **Serwer:** Potężny komputer (lub kilka) trzyma bazę danych (np. MySQL, Oracle). On odpowiada za przechowywanie i przetwarzanie zapytań.
 - **Klient:** Program na komputerze użytkownika (np. aplikacja okienkowa, strona WWW). Klient wysyła zapytanie (np. "pokaż faktury") do serwera, a serwer odsyła tylko wynik.
- **Przykład:** Pracownicy w biurze mają na swoich komputerach program magazynowy, który łączy się z centralnym serwerem SQL w serwerowni.





C. Architektura trójwarstwowa (prezentacja - aplikacja - dane)

Standard w nowoczesnych aplikacjach internetowych.

- **Działanie:** Pomiędzy klientem a bazą danych pojawia się "pośrednik" – serwer aplikacji.
 - **Warstwa 1 (Klient):** Przeglądarka internetowa (tylko wyświetla dane, nic nie wie o bazie).
 - **Warstwa 2 (Aplikacja):** Serwer WWW (np. z kodem PHP, Java, Python). To on zawiera logikę biznesową (np. naliczanie rabatów).
 - **Warstwa 3 (Dane):** Serwer bazy danych.
- **Przykład:** Logujesz się do bankowości internetowej. Przeglądarka (1) wysyła login do serwera aplikacji (2), ten sprawdza poprawność danych, łączy się z bazą (3), pobiera stan konta i wysyła go z powrotem do przeglądarki.



3. Przykłady systemów w różnych architekturach

Architektura	SZBD / Technologia	Przykład zastosowania
Jednowarstwowa	SQLite, Microsoft Access	Aplikacja w smartfonie (np. lista zadań), gdzie dane są przechowywane lokalnie na telefonie.
Dwuwarstwowa	MySQL, PostgreSQL, MS SQL	System sprzedaży w sklepie stacjonarnym. Kasy (klienty) łączą się bezpośrednio z serwerem bazy w zapleczu.
Trójwarstwowa	Oracle (z backendem w Javie), MongoDB (z Node.js)	Portal społecznościowy (np. Facebook). Przeglądarka wyświetla stronę, serwer aplikacji logikę, a baza danych przechowuje posty.
Rozproszona	Cassandra, Google Bigtable	Systemy obsługujące miliardy użytkowników (np. Google, Netflix). Dane są rozrzucone na tysiącach serwerów na całym świecie.

Temat: Instrukcje DDL (ang. Data Definition Language) w MySQL. Create table.

Link do dokumentacji MySQL i tworzenia tabeli:

<https://dev.mysql.com/doc/refman/9.6/en/create-table.html>

1. Trzy podstawowe formy polecenia

-- 1. Normalna definicja tabeli

TEMPORARY tabela istnieje tylko w tej sesji (znika po wylogowaniu)
IF NOT EXISTS – nie rzuca błędu, jeśli tabela już jest.

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] nazwa_tabeli  
    (definicja_kolumn...)  
    [opcje_tabeli]  
    [partycjonowanie];
```

-- 2. Tworzenie tabeli na podstawie zapytania (CTAS)

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] nazwa_tabeli  
    [opcje_tabeli]  
    [partycjonowanie]  
    [IGNORE | REPLACE]  
    AS SELECT ...; # kopiuje strukturę + dane
```

-- 3. Kopiowanie struktury innej tabeli

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] nowa_tabela  
    LIKE stara_tabela; # kopiuje tylko strukturę (kolumny, indeksy, opcje), bez danych.
```

2. Definicja kolumny (definicja_kolumn)

```
nazwa_kolumny typ_danych  
    [NOT NULL | NULL]  
    [DEFAULT wartość_lub_(wyrażenie)]  
    [VISIBLE | INVISIBLE] - domyślnie VISIBLE. Przy zapytaniu SELECT * FROM  
    NAZWA_TABELI, kolumna się nie wyświetli z opcją INVISIBLE  
    [AUTO_INCREMENT]  
    [UNIQUE [KEY]] [[PRIMARY] KEY] -
```

/*

UNIQUE i UNIQUE KEY to to samo. Nie można dodać wartości dla tej kolumny która już istnieje.

- Można mieć **wiele kolumn UNIQUE** w jednej tabeli.
- Kolumna UNIQUE **może mieć NULL** (i w MySQL można mieć wiele NULL-i – bo NULL ≠ NULL).

PRIMARY KEY (lub PRIMARY KEY)

To **główny klucz tabeli**.

Zapewnia:

- ✓ unikalność
 - ✓ brak wartości NULL
 - ✓ automatycznie tworzy indeks
 - ✓ może być tylko jeden PRIMARY KEY w tabeli
- */

```
[COMMENT 'opis']
```

```
[COLLATE zestawienie]
```

/*

określa **jak porównywany i sortowany jest tekst** w kolumnie
COLLATE decyduje m.in.:

- czy wielkość liter ma znaczenie (A = a?)
- czy polskie znaki mają znaczenie (ą = a?)
- jak działa ORDER BY
- jak działa WHERE =
- jak działa LIKE

*/

[GENERATED ALWAYS AS (wyrażenie)]

/*

tworzy **kolumnę generowaną (computed column)**, czyli taką, której wartość jest **automatycznie wyliczana na podstawie innych kolumn**. Nie możesz jej normalnie ustawić w INSERT ani UPDATE (bo jest liczone automatycznie)

Podstawowy przykład

```
CREATE TABLE invoice (  
    price DECIMAL(10,2),  
    quantity INT,  
    total DECIMAL(10,2)  
        GENERATED ALWAYS AS (price * quantity)  
);
```

Teraz:

```
INSERT INTO invoice (price, quantity) VALUES (10.00, 3);
```

*/

[VIRTUAL | STORED]]

/*

VIRTUAL (domyślnie)

```
total DECIMAL(10,2)  
    GENERATED ALWAYS AS (price * quantity) VIRTUAL
```

- ✗ nie zajmuje miejsca w tabeli
- ☒ liczone przy odczycie
- ☒ zawsze aktualne

STORED

```
total DECIMAL(10,2)  
    GENERATED ALWAYS AS (price * quantity) STORED
```

- ☒ zapisuje wartość fizycznie w tabeli
- ☒ może mieć indeks
- ☒ przeliczane przy INSERT/UPDATE

*/

[reference_definition]

/*

Klucz obcy FOREIGN KEY działa tylko w silniku InnoDB

Typ danych klucza obcego musi być identyczny jak w kolumnie referencyjnej

Składnia:

[CONSTRAINT nazwa]

REFERENCES tabela (kolumna)

[ON DELETE akcja]

[ON UPDATE akcja]

Dostępne akcje:

CASCADE

RESTRICT

SET NULL

NO ACTION

1 ON DELETE

◇ a) CASCADE

```
CREATE TABLE users (  
    id INT PRIMARY KEY,  
    name VARCHAR(50)  
);  
  
CREATE TABLE orders (  
    id INT PRIMARY KEY,  
    user_id INT,  
    amount DECIMAL(10,2),  
    FOREIGN KEY (user_id) REFERENCES users(id)  
    ON DELETE CASCADE  
);
```

Działanie:

- Jeśli usuniesz użytkownika, jego zamówienia zostaną usunięte automatycznie.

Przykład:

```
INSERT INTO users VALUES (1, 'Adam');  
INSERT INTO orders VALUES (101, 1, 100);  
INSERT INTO orders VALUES (102, 1, 200);
```

```
DELETE FROM users WHERE id = 1;
```

Efekt:

- users → rekord z id=1 usunięty
- orders → rekordy 101 i 102 też usunięte

◇ b) RESTRICT

```
CREATE TABLE users (  
    id INT PRIMARY KEY,  
    name VARCHAR(50)  
);  
  
CREATE TABLE orders (  
    id INT PRIMARY KEY,  
    user_id INT,  
    FOREIGN KEY (user_id) REFERENCES users(id)  
    ON DELETE RESTRICT  
);
```

```
INSERT INTO users (id, name) VALUES (1, 'Jan'), (2, 'Anna');
INSERT INTO orders (id, user_id) VALUES (100, 1), (101, 2);
```

```
SELECT * FROM users;
SELECT * FROM orders;
```

```
DELETE FROM users WHERE id = 1;
```

```
#1451 - Cannot delete or update a parent row: a foreign key constraint fails (`test`.`orders`,
CONSTRAINT `orders_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`))
```

Działanie:

- Nie pozwala usunąć użytkownika, jeśli istnieją jego zamówienia.

Przykład:

```
DELETE FROM users WHERE id = 1;
```

Efekt:

- ✗ Błąd: nie można usunąć, bo orders odwołuje się do users.id=1.

◊ c) SET NULL

```
CREATE TABLE orders (
  id INT PRIMARY KEY,
  user_id INT NULL,
  FOREIGN KEY (user_id) REFERENCES users(id)
  ON DELETE SET NULL
);
```

Działanie:

- Usunięcie użytkownika ustawia orders.user_id na NULL.

Przykład:

```
DELETE FROM users WHERE id = 1;
```

Efekt:

- users → rekord id=1 usunięty
- orders.user_id → NULL dla wszystkich zamówień tego użytkownika

◊ d) NO ACTION

- W MySQL działa jak RESTRICT (blokuje operację).
- Różnica w standardzie SQL dotyczy kolejności sprawdzania constraintu, ale w praktyce MySQL traktuje to identycznie jak RESTRICT.

```
CREATE TABLE orders (  
    id INT PRIMARY KEY,  
    user_id INT,  
    FOREIGN KEY (user_id) REFERENCES users(id)  
    ON DELETE NO ACTION  
);
```

Efekt:

Nie można usunąć użytkownika, jeśli istnieją zamówienia.

2 ON UPDATE

To działa podobnie, ale zamiast usuwać rekord nadrzędny, zmienia jego wartość (id).

Przykład: zmiana users.id = 1 → 10

◇ a) CASCADE

```
FOREIGN KEY (user_id) REFERENCES users(id) ON UPDATE CASCADE
```

Efekt:

- users.id zmienione → orders.user_id też automatycznie zmienione

Przykład:

```
UPDATE users SET id = 10 WHERE id = 1;
```

- orders.user_id = 1 → automatycznie 10

◇ b) RESTRICT

```
ON UPDATE RESTRICT
```

Efekt:

- Nie można zmienić users.id jeśli istnieją rekordy zależne w orders

```
UPDATE users SET id = 10 WHERE id = 1;
```

- ✗ Błąd: orders.user_id odwołuje się do id=1

◇ c) SET NULL

```
ON UPDATE SET NULL
```

```
CREATE TABLE users (  
    id INT PRIMARY KEY,  
    user_id INT,  
    FOREIGN KEY (user_id) REFERENCES users(id)  
    ON UPDATE SET NULL  
);
```

```

        id INT PRIMARY KEY,

        name VARCHAR(50)

);

CREATE TABLE orders (

        id INT PRIMARY KEY,

        user_id INT NULL,

        FOREIGN KEY (user_id) REFERENCES users(id)

        ON UPDATE SET NULL

);
INSERT INTO users (id, name) VALUES (1, 'Jan');
INSERT INTO orders (id, user_id) VALUES (100, 1);
SELECT * FROM users;
SELECT * FROM orders;

```

users

id	name
1	Jan

orders

id	user_id
100	1

```

UPDATE users SET id = 2
WHERE id = 1;

```

users

id	name
2	Jan

orders

id	user_id
100	NULL

Efekt:

- Zmiana users.id → w tabeli zależnej orders.user_id zostaje ustawione na NULL

```

UPDATE users SET id = 10 WHERE id = 1;

```

- orders.user_id → NULL

◊ d) NO ACTION

```

ON UPDATE NO ACTION
CREATE TABLE users (
        id INT PRIMARY KEY,

```



```

        name VARCHAR(50)
    );

CREATE TABLE orders (
    id INT PRIMARY KEY,
    user_id INT,
    FOREIGN KEY (user_id) REFERENCES users(id)
    ON UPDATE NO ACTION
);

INSERT INTO users (id, name) VALUES (1, 'Jan');
INSERT INTO orders (id, user_id) VALUES (100, 1);
SELECT * FROM users;
SELECT * FROM orders;

```

users

id	name
1	Jan

orders

id	user_id
100	1

```

UPDATE users SET id = 2
WHERE id = 1;

```

✗ Błąd

#1451 - Cannot delete or update a parent row: a foreign key constraint fails (`test`.`orders`, CONSTRAINT `orders\_ibfk\_1` FOREIGN KEY (`user\_id`) REFERENCES `users` (`id`) ON UPDATE NO ACTION)

- Działa jak RESTRICT w MySQL: blokuje zmianę

```

*/
    [CHECK (warunek) [NOT] ENFORCED]
/*

```

Przykład – kontrola wieku

```

CREATE TABLE users (
    id INT PRIMARY KEY,
    age INT,
    CHECK (age >= 18)
);

Teraz:
INSERT INTO users VALUES (1, 25);    -- OK
INSERT INTO users VALUES (2, 15);    -- ✗ BŁĄD

```

ENFORCED jest domyślne – czyli jeśli nie podasz nic, constraint i tak będzie egzekwowany.

```

*/

```

3. Opcje tabeli (table_options) – najważniejsze

```

ENGINE = InnoDB                -- domyślny i zalecany
DEFAULT CHARSET = utf8mb4      -- MUST HAVE dla polskiego
COLLATE = utf8mb4_unicode_ci   -- lub utf8mb4_0900_ai_ci
AUTO_INCREMENT = 1000         -- początkowa wartość

```

```
CREATE TABLE users (

    id INT AUTO_INCREMENT PRIMARY KEY,

    name VARCHAR(50)
) AUTO_INCREMENT = 100;
COMMENT = 'Opis tabeli'
ROW_FORMAT = DYNAMIC                -- zalecane dla InnoDB
/*
```

ROW_FORMAT

Każda tabela InnoDB może mieć różny **format wiersza**:

Format	Co robi
COMPACT	starszy format, krótsze nagłówki, dane długich typów (TEXT, BLOB) mogą być częściowo przenoszone do tzw. overflow page
REDUNDANT	jeszcze starszy, kompatybilny z MySQL 4.x
DYNAMIC	Dynamiczne przechowywanie danych zmiennych typów (VARCHAR, TEXT, BLOB), długie wartości są przechowywane poza wierszem, w tzw. overflow page, pozostaje tylko wskaźnik
COMPRESSED	jak DYNAMIC, ale dodatkowo kompresuje dane, oszczędza miejsce

```
*/
ENCRYPTION = 'Y'                    -- szyfrowanie danych (InnoDB)
DATA DIRECTORY = '/ścieżka/'
/*
```

dotyczy **lokalizacji fizycznego pliku danych tabeli InnoDB (lub MyISAM)**.

- Określa **folder na dysku**, w którym MySQL ma przechowywać pliki tabeli.
- Domyślnie MySQL zapisuje wszystkie pliki w katalogu danych serwera (np. /var/lib/mysql/nazwa_bazy)
- Dzięki DATA DIRECTORY możesz przenieść plik tabeli np. na inny dysk lub partycję.

```
*/
TABLESPACE = nazwa                  -- umieszczenie w tablespace
/*
```

W MySQL **tablespace** to **logiczna przestrzeń w bazie danych**, w której są przechowywane dane tabel i indeksów. MySQL używa tego pojęcia głównie w silniku **InnoDB**, gdzie pozwala na zarządzanie miejscem na dysku w sposób bardziej elastyczny.

- Reprezentuje **plik lub grupę plików na dysku**, w których zapisane są dane i indeksy tabel.
- Pozwala na przechowywanie danych jednej tabeli osobno lub w jednej wspólnej przestrzeni
- Oddziela logicznie zarządzanie danymi (tabele, indeksy) od fizycznego przechowywania (plików .ibd, .idata)

4. W MySQL **[partycjonowanie]** oznacza:

Podział jednej dużej tabeli na **mniejsze, logiczne części (partycje)**, które mogą być przechowywane i przeszukiwane osobno.

- Każda partycja działa jak **podtabela**, ale nadal w ramach jednej tabeli.
- Ułatwia **wyszukiwanie i zarządzanie dużymi danymi**.
- Typy partycjonowania:

Typ	Co robi
RANGE	dzieli wg zakresu wartości (np. data < 2025)
LIST	dzieli wg listy wartości (np. kraj = 'PL', 'DE')
HASH	dzieli wg funkcji hash (np. id MOD 4)
KEY	podobne do HASH, ale używa wbudowanego klucza